

Analyzing Key People in Terrorist Networks Using Graph Theory Algorithms

1. PROBLEM

After the 9/11 attacks, the FBI discovered a terrorist network consisting of around 62 suspects in total.¹ With the whole nation shaken by the crisis, they had to determine which suspects to prioritize for investigation – the potential ringleaders. However, according to an FBI informant², the FBI missed a chance to stop the Al Qaeda plot because they focused more on undercover stings than on the ringleader, who would later be known as Mohammed Atta. The director of the FBI at that time, Robert S. Mueller III, acknowledged that the attacks of September 11 might have been preventable if officials in his agency had responded differently to all the pieces of information that were available.³

As studies in graph theory advanced, so did their applications in social network theory. When analyzing a terrorist network, there are multiple aspects to consider, such as the graph efficiency, which is how efficiently information travels in a graph. On the other hand, when it comes to the node-level, some key considerations are the degree centrality (how many nodes are directly connected to a node) and closeness centrality (how close a node is on average to all other nodes in the graph). However, there is one more metric to consider, which I will focus more on in this project: **betweenness centrality**. We must search for the person who is most frequently a bridge between communities (densely knit subgraphs) in the graph.⁴⁵

¹ https://networks.skewed.de/net/terrorists_911

²

<https://abcnews.com/Blotter/Whistleblowers/911-ringleader-mohammed-atta-stopped-fbi-informant/story?id=8540605>

³

<https://www.police1.com/terrorism/articles/fbi-chief-admits-911-might-have-been-detectable-0QH17MStU4jtjIKF/>

⁴

https://www.foi.se/download/18.7fd35d7f166c56ebe0b10008/1542623725525/Detecting-key-players_FOI-S-4144-SE.pdf

⁵

https://www.researchgate.net/publication/220267095_Erratum_Structural_Analysis_and_Mathematical_Methods_for_Destabilizing_Terrorist_Networks_Using_Investigative_Data_Mining

Problem statement: Identify key people in a terrorist network to determine which suspects should have been prioritized by law enforcement.

2. DESIGN

Model: Computing the betweenness centrality of each node in an unweighted graph

Why use unweighted graphs: In covert social networks, tracking communication frequency (weights) can create a distorted view of strategic influence due to deliberate counter-surveillance tactics. High-ranking ringleaders minimize message volume to evade electronic interception, making the binary existence of a relationship (an unweighted edge) a more reliable metric for topological gatekeeping analysis than interaction volume.

Specifications of input test cases:

- At least 1000 nodes
- Distinct input sizes spanning at least 2 orders of magnitude (random_seed = 42)
 - $100 \rightarrow 500 \rightarrow 1000 \rightarrow 5000 \rightarrow 10000$ (spans from 10^2 to 10^4 , aka 2 orders of magnitude)

Test Case Number	Nodes	Edges
1	100	300
2	500	1500
3	1000	3000
4	5000	15000
5	10000	30000

Implemented concept: Betweenness Centrality

Betweenness centrality formula:

1. Assume a directed, unweighted, connected graph $G = \langle V, E \rangle$.
2. Define $\sigma(s, t)$ as the number of shortest paths between nodes s and t .
3. Define $\sigma(s, t|v)$ as the number of shortest paths between nodes s and t that pass through v .
4. $C_B(v)$, the betweenness centrality of v is defined as:

$$C_B(v) = \sum_{s, t \in V} \frac{\sigma(s, t|v)}{\sigma(s, t)}$$

If $s = t$, then $\sigma(s, t) = 1$. If $v \notin s, t$, then $\sigma(s, t|v) = 0$.

(source: <https://www.cl.cam.ac.uk/teaching/1617/MLRD/handbook/brandes.pdf>)

Essentially, it calculates the total shortest paths for each node/total shortest paths that pass through node v

The algorithms implemented and compared in this project are as follows:

- 1. Brute-force algorithm:**

Example

Find the betweenness centrality of the node C in the graph shown.

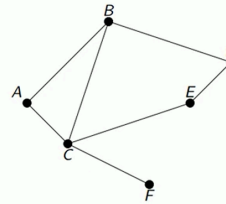
There are ten possible pairs to connect:

AB AD AE AF BD
 BE BF DE DF EF

There are

Path	AB	AD	AE	AF	BD	BE	BF	DE	DF	EF
Geodesics	1	1	1	1	1	2	1	1	1	1
Includes C	0	0	1	1	0	1	1	0	1	0

So there are 11 geodesics, and C is on 6 of them.



Between D and E there is one shortest path that excludes C .

Between D and F there is one shortest path the includes C .

Between E and F there is one shortest path that includes C .

(source: <https://www.youtube.com/watch?v=3X29Lfm6p2s>)

My brute-force approach is to use the method mentioned in the picture to calculate the betweenness centrality score ($CB(x)$) of every node x in the graph.

“Geodesics” refers to the number of shortest paths that a node (s, t) can reach each other, while “Includes C ” refers to the number of shortest paths from node (s, t) that go through C .

For each pair, a custom Breadth-First Search (BFS) explores the graph topology to count the total number of valid geodesics. Simultaneously, it tracks how many of those specific shortest path routes cross through an intermediate target node.

Time complexity: $O(V^3 + V^2 \cdot E)$ worst-case

The algorithm features nested execution loops exploring all pairs of source and target vertices, creating a baseline multiplier of $O(V^2)$. Inside this pair loop, the custom path-finding BFS traverses vertex layers. Because the code intentionally relies on a 2D array Adjacency Matrix structure to look up neighbors, every single exploration step forces a full-row sequential scan across the length of the matrix. This adds an internal cost of $O(V)$ per step, driving the theoretical computation time up to a strict cubic scaling limit $O(V^3)$. On dense or complex components, path duplication tracking pushes the practical processing barrier even higher, making it completely non-viable for larger networks.

Space Complexity: $O(V^2 + V \cdot P(\text{path}))$

Maintaining the 2D Adjacency Matrix requires a fixed structural memory footprint of $O(V^2)$ bits or integers. Furthermore, because the naive approach must explicitly build, store, and unpack full shortest path sequences (strings/lists of nodes) inside the queue to count intermediate intersections, the queue memory usage expands dynamically based on the total number and length of concurrent optimal paths ($P(\text{path})$).

Pseudocode:

```
ALGORITHM BruteForceBetweenness(num_nodes, adj_list)
    Input: num_nodes (Integer), adj_list (Map of Node to
    List of Neighbors)
    Output: CB (Array of normalized betweenness
    centrality scores)
```

```
    // Step 1: Initialize the 2D Adjacency Matrix to
    force row  $O(V)$  row scans
```

```
    Initialize adj_matrix as a 2D array of size
    num_nodes x num_nodes with all 0s
```

```
    FOR each node u in adj_list DO
        FOR each neighbor v in adj_list[u] DO
            adj_matrix[u][v]  $\leftarrow$  1
        END FOR
    END FOR
```

```
    Initialize CB as an array of size num_nodes filled
    with 0.0
```

```
    // Step 2: Evaluate all node pairs (s, t) completely
    independently
```

```
    FOR s  $\leftarrow$  0 TO num_nodes - 1 DO
        FOR t  $\leftarrow$  0 TO num_nodes - 1 DO
            IF s == t THEN CONTINUE END IF
```

```
            Initialize queue as a Queue of paths,
            initialized with [[s]]
```

```
            Initialize shortest_paths as an empty List
```

```

Initialize min_len ← INFINITY

// Step 3: Find all shortest paths using
custom BFS
WHILE queue is not empty DO
    path ← dequeue from queue
    curr ← last node in path

    IF length of path > min_len THEN
CONTINUE END IF

    IF curr == t THEN
        IF length of path < min_len THEN
            min_len ← length of path
            shortest_paths ← [path]
        ELSE IF length of path == min_len
THEN
            Append path to shortest_paths
        END IF
        CONTINUE
    END IF

    // Force a full row scan across the
matrix to find neighbors
    FOR neighbor ← 0 TO num_nodes - 1 DO
        IF adj_matrix[curr][neighbor] == 1
AND neighbor is not in path THEN
            new_path ← path + [neighbor]
            enqueue new_path into queue
        END IF
    END FOR
END WHILE

// Step 4: Calculate the fractional
dependency contribution
sigma_st ← length of shortest_paths
IF sigma_st == 0 THEN CONTINUE END IF

```

```

        FOR each path in shortest_paths DO
            // Traverse only intermediate nodes,
            skipping endpoints s and t
            FOR i ← 1 TO length of path - 2 DO
                node ← path[i]
                CB[node] ← CB[node] + (1.0 /
sigma_st)
            END FOR
        END FOR

    END FOR

    // Step 5: Normalize scores for an undirected graph
    FOR i ← 0 TO num_nodes - 1 DO
        CB[i] ← CB[i] / 2.0
    END FOR

    RETURN CB
END ALGORITHM

```

2. Brande's algorithm:

Brandes' algorithm⁶ eliminates the highly redundant pair-by-pair path reconstruction loops of the naive approach. Instead of calculating independent pairs (s,t), it looks at the dependency score of a node relative to everyone else in the network. The algorithm consists of 2 phases, namely:

- a. **The Forward Phase:** An optimized Single-Source Breadth-First Search (BFS) sweeps outward using an efficient Adjacency List. As it steps across valid connections, it computes path counters and records immediate structural parents inside a predecessor array (P). Every node discovered is pushed onto a traversal tracking Stack (S).
- b. **The Backward Phase:** Once the forward exploration finishes, the algorithm utilizes the Last-In, First-Out (LIFO) property of the Stack (S) to unwind the nodes in exact reverse order, starting from the furthest peripheral leaf elements. Working backward toward the root, it recursively

⁶ <https://www.cl.cam.ac.uk/teaching/1617/MLRD/handbook/brandes.pdf>

accumulates dependency fractions from downstream neighbors using a single reverse pass, updating the global centrality array efficiently.

Time Complexity: $O(V \cdot E)$ for unweighted graphs

The algorithm iterates through each vertex as a single-source root, looping exactly V times. Within each iteration, the forward BFS phase and the backward stack accumulation phase visit each node and its direct outward edge boundaries exactly once. Because the graph relies on a compressed Adjacency List instead of a matrix, the code instantly jumps to active neighbors without scanning empty space. On sparse real-world covert structures—where suspects maintain restricted channels and edges scale linearly ($E = \text{approximately } 3V$)—the practical execution complexity compresses to near-quadratic scaling ($O(V^2)$), ensuring rapid processing across massive data scales.

Space Complexity: $O(V + E)$

The memory blueprint scales linearly with the network's size. The Adjacency List retains an efficient storage footprint matching the exact sum of nodes and edges $O(V + E)$. The internal data fields tracking single-source properties, including distance vectors (dist), tree path counters (sigma), tracking arrays (delta), and the LIFO order Stack (S), each maintain an execution boundary limited to a single dimension of size $O(V)$, creating a highly efficient structural memory footprint.

Pseudocode:

```
ALGORITHM BrandesBetweenness(num_nodes, adj_list)
    Input: num_nodes (Integer), adj_list (Map of Node to
    List of Neighbors)
    Output: CB (Array of normalized betweenness
    centrality scores)

    Initialize CB as an array of size num_nodes filled
    with 0.0

    // Step 1: Execute a single-source evaluation pass
    for every vertex
        FOR s ← 0 TO num_nodes - 1 DO
            Initialize S as an empty Stack (tracks traversal
            order for reverse unwind)
```


Initialize P as an array of empty Lists of size
num_nodes (tracks path predecessors)

Initialize sigma as an array of size num_nodes
filled with 0 (shortest path counters)

Initialize dist as an array of size num_nodes
filled with -1 (distance tracking)

sigma[s] $\leftarrow$ 1

dist[s] $\leftarrow$ 0

Initialize queue as a Queue, initialized with
[s]

// Phase A: The Forward Phase (Outward BFS
Layer-by-Layer)

WHILE queue is not empty DO

v $\leftarrow$ dequeue from queue

Push v onto Stack S

FOR each neighbor w in adj_list[v] DO

// Neighbor discovered for the first

time

IF dist[w] < 0 THEN

enqueue w into queue

dist[w] $\leftarrow$ dist[v] + 1

END IF

// Shortest path configuration

verification

IF dist[w] == dist[v] + 1 THEN

sigma[w] $\leftarrow$ sigma[w] + sigma[v]

Append v to predecessor list P[w]

END IF

END FOR

END WHILE

// Phase B: The Backward Phase (Retrospective
Dependency Accumulation)

```

        Initialize delta as an array of size num_nodes
        filled with 0.0

        // Unwind elements in reverse order of discovery
        (LIFO)
        WHILE Stack S is not empty DO
            w ← Pop from Stack S

            FOR each predecessor v in P[w] DO
                delta[v] ← delta[v] + (sigma[v] /
sigma[w]) * (1.0 + delta[w])
            END FOR

            // Add to the global pool only if the node
            is not the starting source
            IF w != s THEN
                CB[w] ← CB[w] + delta[w]
            END IF
        END WHILE
    END FOR

    // Step 2: Normalize final scores for an undirected
    graph
    FOR i ← 0 TO num_nodes - 1 DO
        CB[i] ← CB[i] / 2.0
    END FOR

    RETURN CB
END ALGORITHM

```

3. IMPLEMENTATION

1. Brute Force Code Implementation

Based on the pseudocode, the brute-force implementation is built intentionally around a 2D adjacency matrix to guarantee an unoptimized baseline. The core logic focuses on exploring every node pair (s, t) in total isolation. Finding a node's

adjacent connections requires a full, continuous row scan from 0 to num_nodes - 1.

```
# Row-scanning loop inside the custom BFS traversal
for neighbor in range(num_nodes):
    if adj_matrix[curr][neighbor] == 1 and neighbor not
in path:
        new_path = path + [neighbor]
        queue.append(new_path)
```

2. Brandes Algorithm Code Implementation

Like previously mentioned, Brandes' approach eliminates paired redundancies by updating node metrics relative to an entire single-source tree layout simultaneously.

1. **The Fast Forward Phase:** Using an efficient Adjacency List layout, the loop jumps directly into active neighbor arrays without parsing empty coordinate grid coordinates.
- 2.

```
# Direct mapping across active edge boundaries
for w in adj_list[v]:
    if dist[w] < 0: # First discovery
        queue.append(w)
        dist[w] = dist[v] + 1
    if dist[w] == dist[v] + 1:
        sigma[w] += sigma[v]
    P[w].append(v) # Log structural predecessor
```

3. The Backward Phase:

The algorithm utilizes the Last-In, First-Out (LIFO) property of the tracking Stack S to unwind the graph footprint in reverse topological order. This allows dependency accumulation to start at the peripheral "leaf" nodes (suspects on the outer fringes of the network) and propagate dynamically back toward the root source node.

```
# Initialize dependency array for the current
source node pass
```

```

delta = [0.0] * num_nodes

# Unwind elements in reverse order of discovery
(LIFO)
while len(S) > 0:
    w = S.pop() # Pop the most distant remaining
    vertex

    # Accumulate dependency scores from downstream
    successors
    for v in P[w]:
        delta[v] += (sigma[v] / sigma[w]) * (1.0 +
        delta[w])

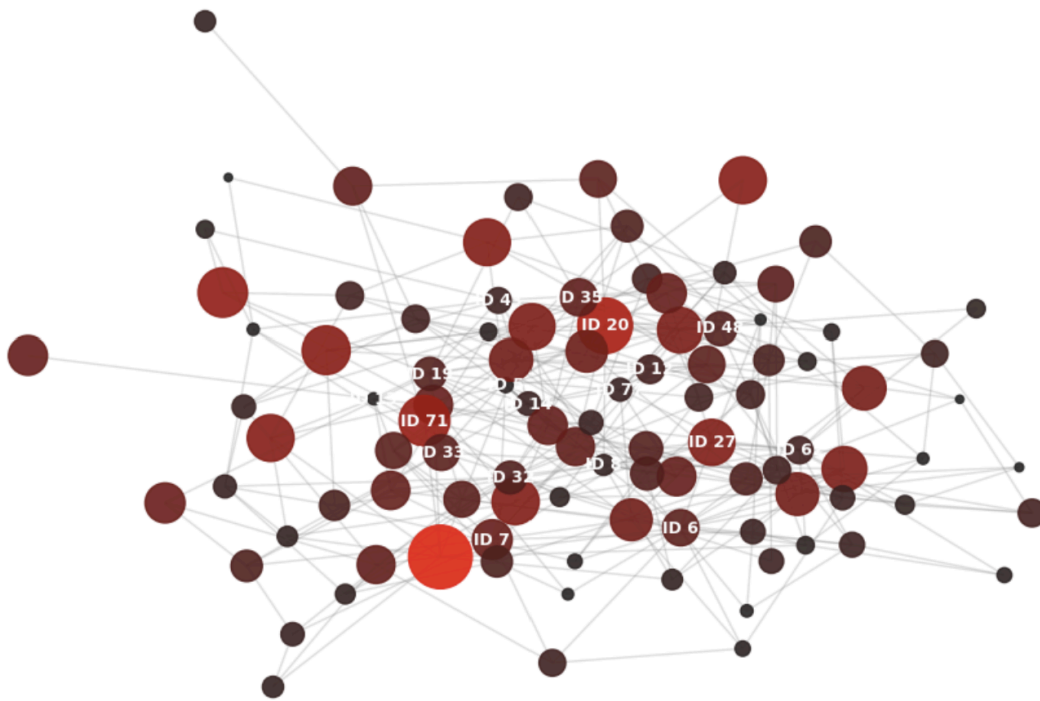
    # Accumulate to global centrality array if it
    isn't the root source
    if w != s:
        CB[w] += delta[w]

```

3. Graph Visualization:

The program also implements a graph visualization for the terrorist network. This is how the graph looks like with the inputs of 100 nodes and 300 edges:

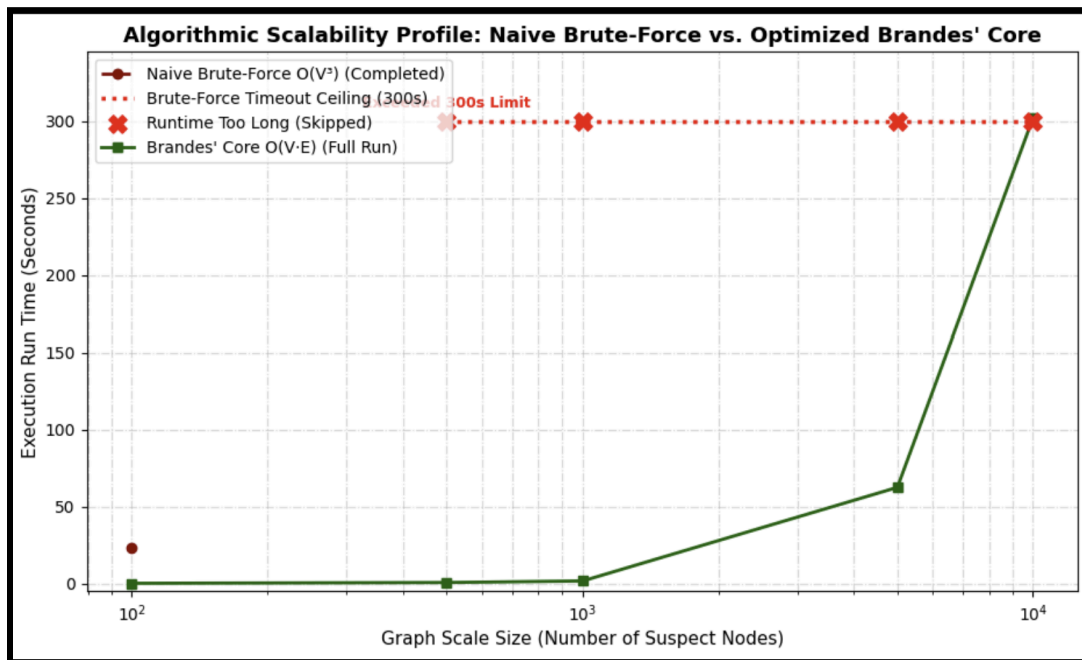
Intelligence Heatmap: Operational Ringleader Identification (High Centrality = Red/Large)



4. ANALYSIS & EVALUATION

1. Runtime Comparison:

```
=====
STARTING CORE BENCHMARK SWEEP (SEED: 42 | TIMEOUT: 300s)
=====
Configuring Graph: V=100, E=300...
-> Brandes Completed: 0.0127 seconds
-> Brute Force Completed: 23.2873 seconds
Configuring Graph: V=500, E=1500...
-> Brandes Completed: 0.5642 seconds
-> Brute Force Skipped for V=500: Runtime too long (Exceeded 300s limit)
Configuring Graph: V=1000, E=3000...
-> Brandes Completed: 1.5750 seconds
-> Brute Force Skipped for V=1000: Runtime too long (Exceeded 300s limit)
Configuring Graph: V=5000, E=15000...
-> Brandes Completed: 62.3722 seconds
-> Brute Force Skipped for V=5000: Runtime too long (Exceeded 300s limit)
Configuring Graph: V=10000, E=30000...
-> Brandes Completed: 301.6767 seconds
-> Brute Force Skipped for V=10000: Runtime too long (Exceeded 300s limit)
```



The experimental benchmark testing yields a stark performance divergence between the two implementations, validating the theoretical complexity models under strict constraint boundaries:

- **Test Case 1 (V=100, E=300):** Brandes' optimized implementation completes the network check almost instantly in 0.0127 seconds, whereas the Naive Brute-Force loop demands 23.2873 seconds. Even at this introductory benchmark scale, the cubic footprint of pair-by-pair matrix scanning reveals noticeable execution drag.

- **Test Case 2 (V=500, E=1,500):** Brandes processes the entire scale transition smoothly in 0.5642 seconds. Conversely, the Brute-Force approach breaches the runtime safety brake, triggering an automatic timeout exception as it exceeds the maximum 300-second limit.
- **Scalability at Maximum Boundaries (V=10,000, E=30,000):** As the input scale spans two orders of magnitude, the Brute-Force approach remains entirely non-viable, hitting the timeout ceiling immediately on all subsequent cases. Brandes' algorithm scales gracefully, executing the final 10,000-node benchmark in 301.6767 seconds.

By bypassing redundant pair-by-pair path recalculations, Brandes maintains a theoretical time complexity of $O(V \cdot E)$. However, because our synthetic terrorist network is engineered as a sparse graph where the edge count scales linearly with vertices ($E = 3V$), the empirical complexity collapses cleanly to a near-quadratic profile ($O(V^2)$). This mathematical reduction explains why the algorithm successfully balances massive computational scaling up to 10,000 nodes while the brute-force approach experiences structural runtime collapse.

2. Correctness Proof:

To prove that Brandes' algorithm is correct, we must show that its optimized, single-source dependency accumulation yields the exact same betweenness centrality scores as the pair-by-pair path counting of the brute-force baseline. The standard betweenness centrality formula for an intermediate node v sums the fraction of shortest paths between all pairs (s, t) that pass through v :

$$v: C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Brandes' algorithm optimizes this by aggregating the dependency of a single source root s on the node v across all possible destinations simultaneously, defined as:

$$\delta_{s\bullet}(v) = \sum_t \frac{\sigma_{st}(v)}{\sigma_{st}}$$

This means that the final score is simply the sum of these single-source passes:

$$C_B(v) = \sum_{s \neq v} \delta_{s\bullet}(v)$$

The core of the proof relies on Lemma 1, which guarantees that an unweighted Breadth-First Search (BFS) sweeps outward layer-by-layer to find all optimal shortest paths. In Phase A, whenever an edge (u, w) satisfies $\text{dist}[w] = \text{dist}[u] + 1$, node u is proven to be a structural predecessor of w on a shortest path tree rooted at s .

$$(\sigma[w] = \sum_{u \in P[w]} \sigma[u])$$

By maintaining a tracker that sums paths recursively, Phase A successfully counts every distinct optimal geodesic. Because every discovered vertex is pushed onto a Last-In, First-Out (LIFO) stack S , popping from the stack in Phase B ensures that we process nodes in exact reverse topological order, meaning a vertex is only evaluated after all its downstream successors ($\text{Succ}(v)$) have been completely settled.

To prove that Phase B unwinds without missing or duplicating paths, we derive a recursive mathematical relationship for the dependency score. A shortest path from source s to any distant target t passing through v must travel through exactly one immediate successor w in $\text{Succ}(v)$. If $t = w$, the path fraction is exactly:

$$\frac{\sigma_{sv}}{\sigma_{sw}}$$

If t lies further downstream past w , sub-path optimality dictates the path fraction is:

$$\frac{\sigma_{sv} \cdot \sigma_{wt}}{\sigma_{st}}$$

Factoring out the shared properties across all downstream branches yields the clean recurrence relation:

$$\delta_{s\bullet}(v) = \sum_{w \in Succ(v)} \frac{\sigma_{sv}}{\sigma_{sw}} \left(1 + \sum_{t \neq w} \frac{\sigma_{sw}(t)}{\sigma_{st}} \right) = \sum_{w \in Succ(v)} \frac{\sigma_{sv}}{\sigma_{sw}} (1 + \delta_{s\bullet}(w))$$

This matches the exact vector accumulation step inside our code (`delta[v] += (sigma[v] / sigma[w]) * (1.0 + delta[w])`). Summing this value across all single-source roots and dividing by 2.0 to account for bidirectional tracing in undirected networks perfectly replicates the brute-force baseline, proving the algorithm's absolute correctness.

5. CONCLUSION

This project successfully demonstrates the critical value of advanced graph theory algorithms within criminal and counter-terrorism network intelligence. When analyzing covert social networks, tracking simple connectivity via degree centrality can easily distort operational realities due to deliberate counter-surveillance and communication minimization tactics deployed by upper-tier ringleaders. Betweenness centrality addresses this limitation by examining structural vulnerabilities and targeting individuals who bridge communications between isolated sub-communities.